

Ingesta y Análisis de datos con Kafka y NiFi

Adrián Robles Arques

Introducción

En el presente proyecto vamos a trabajar a partir de los datos generados en los sprints anteriores, donde creamos, siguiendo diferentes procesos, tres tablas de datos gestionadas por Hive y almacenadas en el sistema distribuido de HDFS. Con dichos datos, vamos a generar una nueva tabla cruzando datos de las tres y aplicando ciertas restricciones. Con dichos resultados, generaremos un flujo de datos en Nifi para suministrarlos a un topic de Kafka y crear una nueva tabla, esta vez propia de Hive.

Consideraciones previas

Para este proyecto, vamos a crear un consumidor de datos de Kafka que se recibirá los datos que se procesen a través del flujo de datos diseñado en Nifi. Para ello, vamos a generar los datos que deben ser procesados mediante Apache Hive, realizando una unión de las tres tablas para extraer los datos solicitados.

Los formatos de las diferentes tablas ya han sido facilitados en los 3 sprints previos, y los que necesitamos en este caso concreto serán especificados en la sección del informe donde crearemos la tabla. Prestaré especial atención en las fases novedosas respecto a los sprints previos.

Procedimiento

En esta sección desarrollaré los pasos para implementar los procesos correspondiente a las fases 4 y 5, dado que ambas están fuertemente relacionadas y me parece más indicado explicarlas seguidamente, tal como las he llevado a cabo, que por separado en sesiones independientes.

Arranque de sistemas y comprobaciones iniciales

Vamos a iniciar los sistemas necesarios para el desarrollo de las tareas siguientes, en primer lugar, arrancaremos HDFS para acceder a los datos almacenados de las tres tablas creadas en los sprints anteriores, empleando los comandos que ya conocemos:

```
bigdata@Big-Data:~$ cd /home/bigdata/hadoop-3.3.6/
bigdata@Big-Data:~/hadoop-3.3.6$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [Big-Data]
```

A continuación vamos a arrancar Kafka sin el entorno de Kafka connect, emplearemos solo el ecosistema Kafka con Zookeeper para proceder a la ingesta de datos que procederán desde el flujo que diseñemos en Nifi. Para ello empleamos el comando 'confluent local services kafka start'.

```
bigdata@Big-Data:~/confluent-7.5.1$ confluent local services kafka start
The local commands are intended for a single-node development environment only, NOT for production u
sage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.011580
ZooKeeper is [UP]
Starting Kafka
Kafka is [UP]
bigdata@Big-Data:~/confluent-7.5.1$
```

Seguidamente creamos un consumidor no Avro, dado que en este caso vamos a suministrar datos en formato json estándar, tras realizar una conversión desde el formato parquet en el que se almacenan en HDFS. Para ello empleamos el comando 'bin/kafka-console-consumer', seguido de la indicación del servidores a emplear '--bootstrap-server localhost:9092' y el nombre del topic con '--topic aggregation'.

```
bigdata@Big-Data:~/confluent-7.5.1$ bin/kafka-console-consumer --bootstrap-server localhost:9092 --topic
aggregation
[2025-05-30 13:57:31,168] WARN [Consumer clientId=console-consumer, groupId=console-consumer-24331] Erro
r while fetching metadata with correlation id 2 : {aggregation=LEADER_NOT_AVAILABLE} (org.apache.kafka.c
lients.NetworkClient)
```

Después iniciamos los servicios de Apache NiFi, colocándonos en el directorio de instalación y empleando el comando 'bin/nifi.sh start'.

```
bigdata@Big-Data:~/confluent-7.5.1$ cd /home/bigdata/nifi-1.23.2/
bigdata@Big-Data:~/nifi-1.23.2$ bin/nifi.sh start

Java home: /usr/lib/jvm/java-8-openjdk-amd64
NiFi home: /home/bigdata/nifi-1.23.2

Bootstrap Config File: /home/bigdata/nifi-1.23.2/conf/bootstrap.conf

bigdata@Big-Data:~/nifi-1.23.2$
```

Además, como norma de buenas prácticas, introducimos el comando para visualizar las últimas 400 líneas de logs de la aplicación con el comando 'tail -400f /home/bigdata/nifi-1.23.2/logs/nifi-app.log'.

```
bigdata@Big-Data:~/nifi-1.23.2$ tail -400f /home/bigdata/nifi-1.23.2/logs/nifi-app.log
2025-05-30 13:45:07,761 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Initiating checkpoint of FlowFile Repository
2025-05-30 13:45:07,853 INFO [pool-7-thread-1] o.a.n.c.r.WriteAheadFlowFileRepository Successfully checkpointed FlowFile Repository with 0 records in 0 milliseconds
2025-05-30 13:45:07,888 INFO [Cleanup Archive for default] o.a.n.c.repository.FileSystemRepository Successfully deleted 0 files (0 bytes) from archive
2025-05-30 13:45:07,911 INFO [Cleanup Archive for default] o.a.n.c.repository.FileSystemRepository Archive cleanup completed for container default; will now allow writing to this container. Bytes used = 22,08 GB, bytes free = 6,79 GB, capacity = 28,87 GB
```

Por último entramos en Hive acudiendo igualmente al directorio de instalación y ejecutando la aplicación mediante el comando 'bin/beeline -u jdbc:hive2://'

```
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://>
```

Una vez en Hive, realizamos una última comprobación antes de iniciar el desarrollo de las fases subsiguientes, comprobando para ello que las tres tablas que emplearemos siguen siendo accesibles:

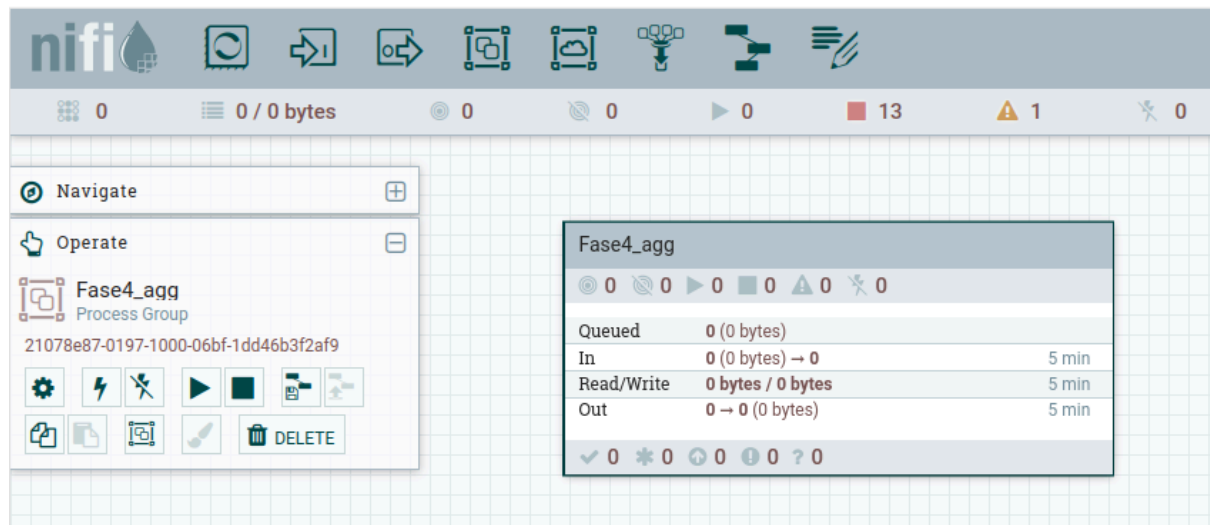
```
+-----+
|  tab_name  |
+-----+
| aerolineas |
| aeropuertos |
| fruits     |
| rutas      |
| shops      |
+-----+
```

Estas herramientas se han desplegado en terminales independientes, de forma que se pueda trabajar en paralelo en cada una de ellas.

Creación del flujo de datos en NiFi

Una vez desplegadas las herramientas necesarias y realizadas las comprobaciones previas, vamos a proceder con la creación del proceso en NiFi necesario para leer los datos de la tabla que generaremos en Hive a partir de las otras tres almacenadas en HDFS, transformar los datos de formato parquet a Json, y posteriormente suministrarlos al consumidor de Kafka en el topic 'aggregations'.

Comenzamos creando un nuevo grupo de procesadores para esta fase:



Para realizar la transformación necesaria tal como la he descrito anteriormente, vamos a necesitar los siguientes procesadores con la configuración indicada:

- **ListHDFS:** Procesador necesario para acceder a los datos tipo parquet almacenados en el sistema distribuido de archivos.

Configure Processor | ListHDFS 1.23.2

Invalid

SETTINGS SCHEDULING **PROPERTIES** RELATIONSHIPS COMMENTS

Required field

Property	Value
Hadoop Configuration Resources	/home/bigdata/hadoop-3.3.6/etc/core-site.xml, /home/bi...
Kerberos Credentials Service	No value set
Kerberos User Service	No value set
Kerberos Principal	No value set
Kerberos Keytab	No value set
Kerberos Password	No value set
Kerberos Relogin Period	4 hours
Additional Classpath Resources	No value set
Directory	/data/aggregations
Recurse Subdirectories	true
Record Writer	No value set
File Filter	['\.*']

CANCEL APPLY

- **FetchParquet:** Este procesador permite convertir archivos de formato parquet a una serie diferente de formatos. En este caso seleccionaremos el Writer para formato Json, que requerirá crear el controlador apropiado JsonRecordSetWriter.

Configure Processor
FetchParquet 1.23.2

Invalid

SETTINGS
SCHEDULING
PROPERTIES
RELATIONSHIPS
COMMENTS

Required field

Property	Value
Hadoop Configuration Resources	/home/bigdata/hadoop-3.3.6/etc/core-site.xml, /home/bl...
Kerberos Credentials Service	No value set
Kerberos User Service	No value set
Kerberos Principal	No value set
Kerberos Keytab	No value set
Kerberos Password	No value set
Kerberos Relogin Period	4 hours
Additional Classpath Resources	No value set
Filename	\${path}/\${filename}
Record Writer	JsonRecordSetWriter

CANCEL
APPLY

- **LogAttribute:** Aunque el uso de este procesador no sea necesario para la transformación del formato de los datos, vamos a emplearlo para guardar los logs del proceso, lo cual es una buena práctica.

Configure Processor
LogAttribute 1.23.2

Stopped

SETTINGS
SCHEDULING
PROPERTIES
RELATIONSHIPS
COMMENTS

Required field

Property	Value
Log Level	info
Log Payload	true
Attributes to Log	No value set
Attributes to Log by Regular Expression	.*
Attributes to Ignore	No value set
Attributes to Ignore by Regular Expression	No value set
Log FlowFile Properties	true
Output Format	Line per Attribute
Log prefix	No value set
Character Set	UTF-8

CANCEL
APPLY

- **PublishKafka_2.6:** Finalmente, empleamos este procesador para que tome los datos ya en formato Json y los publique para el consumidor de Kafka, empleando el topic 'aggregation'.

Configure Processor | PublishKafka_2_6 1.23.2

Invalid

SETTINGS

SCHEDULING

PROPERTIES

RELATIONSHIPS

COMMENTS

Required field

Property	Value
Kafka Brokers	localhost:9092
Topic Name	aggregations
Use Transactions	true
Transactional Id Prefix	No value set
Message Demarcator	No value set
Failure Strategy	Route to Failure
Delivery Guarantee	Guarantee Replicated Delivery
Attributes to Send as Headers (Regex)	No value set
Message Header Encoding	UTF-8
Security Protocol	PLAINTEXT
SASL Mechanism	GSSAPI
Kerberos Credentials Service	No value set

CANCEL

APPLY

Como hemos indicado previamente, adicionalmente a los procesadores indicados, vamos a emplear el controlador **JsonRecordSetWriter** para establecer el formato que usará el procesador **FetchParquet** para realizar la conversión del formato.

Configure Controller Service | JsonRecordSetWriter 1.23.2

SETTINGS

PROPERTIES

COMMENTS

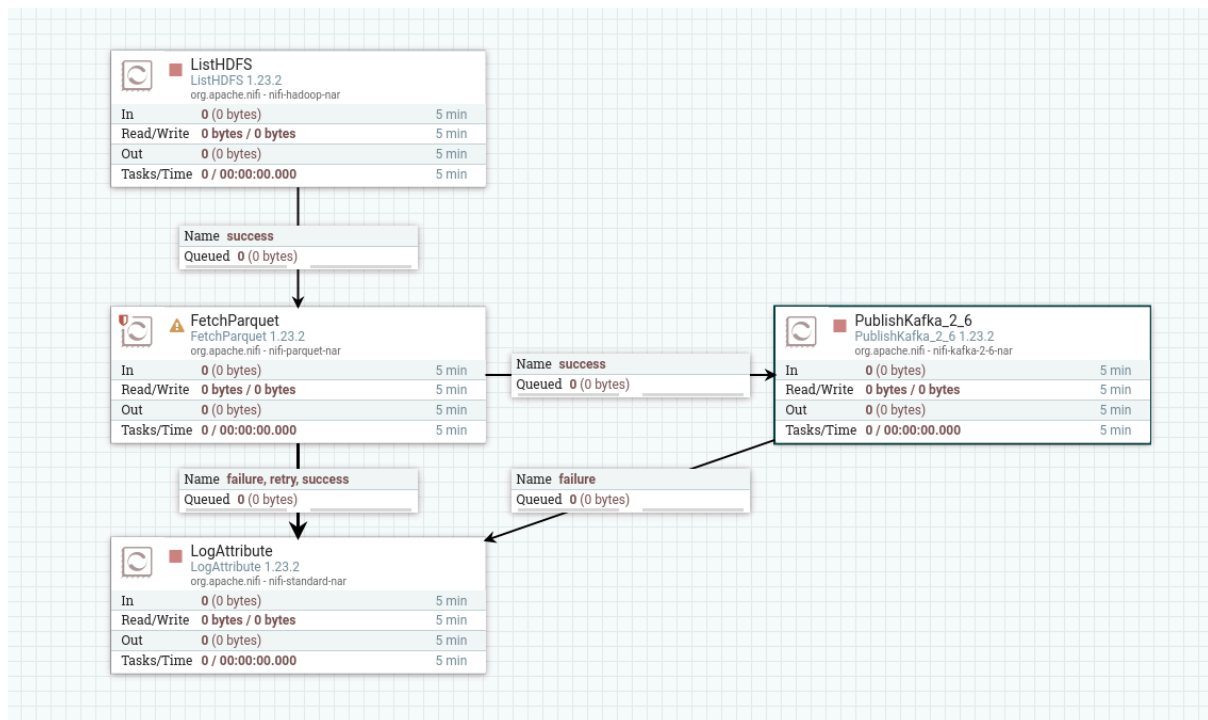
Required field

Property	Value
Schema Write Strategy	Set 'schema.name' Attribute
Schema Cache	No value set
Schema Access Strategy	Inherit Record Schema
Date Format	No value set
Time Format	No value set
Timestamp Format	No value set
Pretty Print JSON	false
Suppress Null Values	Never Suppress
Output Grouping	Array
Compression Format	none

CANCEL

APPLY

Finalmente obtenemos el siguiente diseño de flujo:



Establecemos las relaciones entre procesadores tal como se ve en la imagen, guardando en logs todos los procedimientos de transformación de parquet a Json, así como las publicaciones fallidas en Kafka.

Como se puede ver, el procesador **FetchParquet** todavía no está operativo, dado que antes debemos arrancar el controlador del que depende su funcionamiento.

Enable Controller Service

Service

JsonRecordSetWriter

Steps To Enable JsonRecordSetWriter

Enabling this controller service

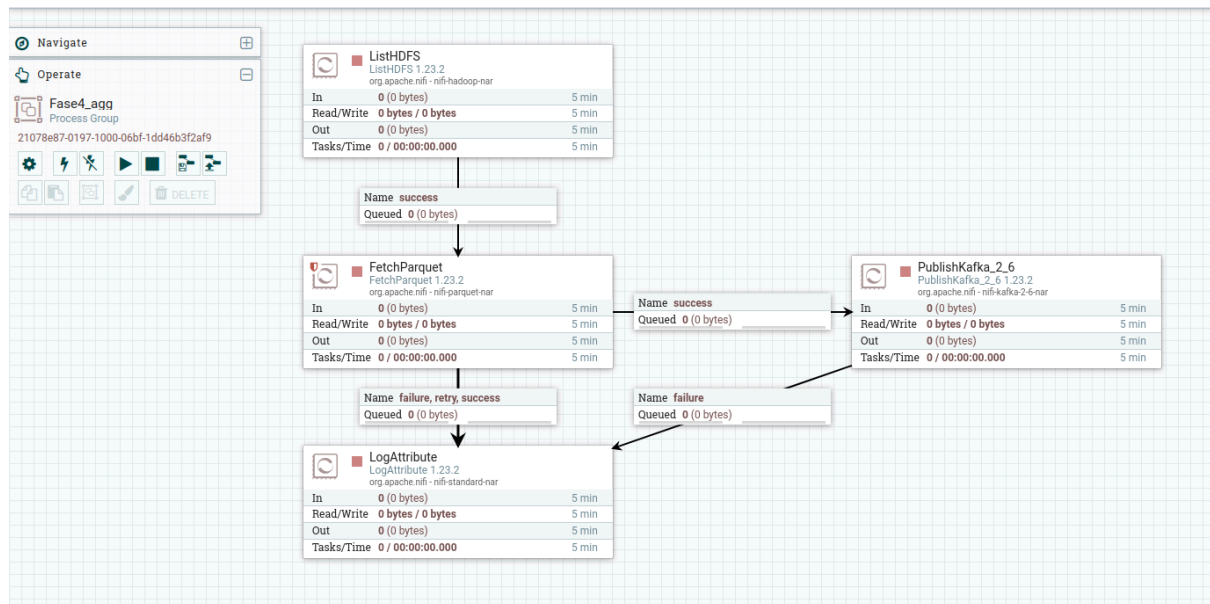
Referencing Components

Processors (1)

FetchParquet FetchParquet

CLOSE

Después de arrancar el controlador ya podemos iniciar el flujo y dejarlo disponible para el consumidor de Kafka, a la espera de que generemos la nueva tabla de datos agregados a partir de Hive:



Creación de la tabla de datos agregados en Hive

Con el flujo de procesos de NiFi ya listo, vamos a crear la tabla agregada a partir de los datos de las otras tres tablas. Para ello deberemos cruzar datos de la ID de la aerolínea que opera la ruta, procedente de la tabla 'rutas', con las IDs que aparecen en la tabla de 'aerolíneas', así como la identificación del aeropuerto que aparece en la tabla 'aeropuertos' con la que referencia la tabla 'rutas'.

A partir de los datos cruzados de las tres tablas, extraeremos el total de rutas de aerolíneas inactivas que toman como destino aeropuertos situados a una altitud superior a 5000 pies. La tabla generada la guardaremos en formato parquet en la ubicación designada para la lectura del procesador de HDFS: '/data/aggregations'.

Para crear esta tabla gestionada por Hive vamos a emplear la siguiente query de HQL:

```
0: jdbc:hive2://> CREATE TABLE aggregations
> STORED AS parquet
> LOCATION '/data/aggregations'
> AS SELECT COUNT(rutas.destinationairport) AS total_destinations, aeropuertos.country AS country
> FROM rutas
> JOIN aerolineas ON rutas.airlineid = aerolineas.airlineid
> JOIN aeropuertos ON aeropuertos.airportid = rutas.destinationid
> WHERE rutas.stops = 0 AND aeropuertos.altitude > 5000 AND aerolineas.active = False
> GROUP BY aeropuertos.country;
```


Comprobamos a continuación que se ha creado la tabla en Hive:

```
0: jdbc:hive2://> SHOW TABLES;
OK
+-----+
| tab_name |
+-----+
| aerolineas |
| aeropuertos |
| aggregations |
| fruits |
| rutas |
| shops |
| test |
+-----+
7 rows selected (1,014 seconds)
0: jdbc:hive2://>
```

Y vemos los resultados que devuelve:

```
0: jdbc:hive2://> SELECT * FROM aggregations;
25/06/02 13:59:20 [a3946730-d23e-4050-aef8-99116103a980 main]
OK
+-----+-----+
| aggregations.total_destinations | aggregations.country |
+-----+-----+
| 1 | Indonesia |
| 10 | Kenya |
| 23 | Mexico |
+-----+-----+
3 rows selected (0,185 seconds)
0: jdbc:hive2://>
```

Y vemos que los datos aparecen en el explorador de HDFS

Browse Directory

Show entries

Search:

☐	Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name	
☐	-rw-r--r--	bigdata	supergroup	541 B	May 30 14:39	1	128 MB	000000_0	

Showing 1 to 1 of 1 entries

Previous

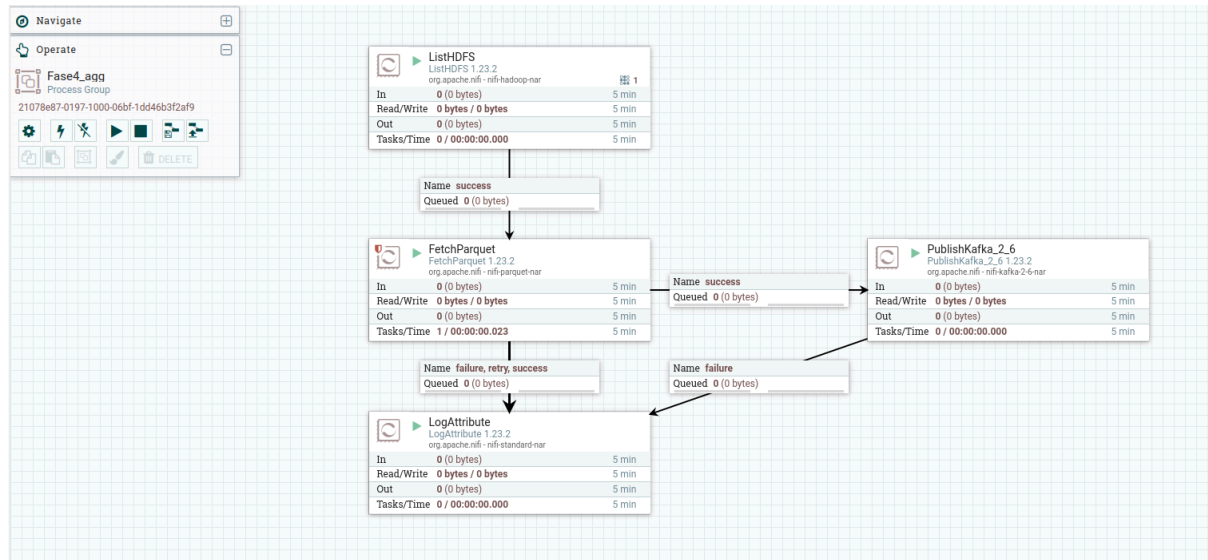
1

Next

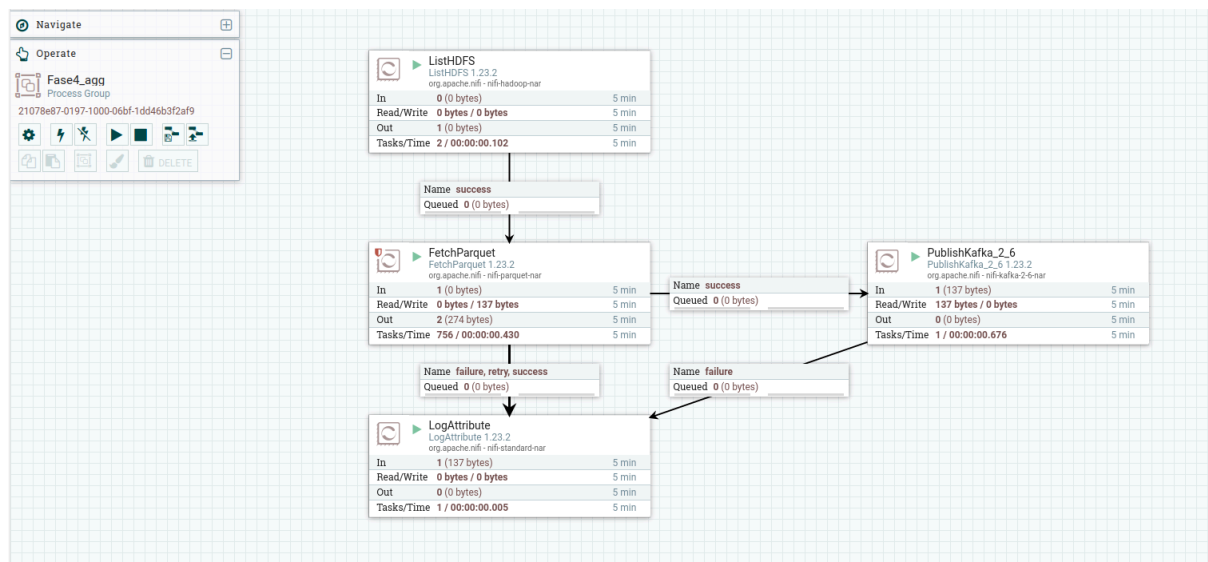
Con la tabla ya creada, solo queda activar el grupo de procesos previamente configurado en NiFi para que lea los datos almacenados en ‘/data/aggregations’ en formato parquet, los transforme en Json y los transfiera al consumidor de Kafka.

Ejecución del grupo de procesos de NiFi

Con la tabla de datos en HDFS en la ubicación donde el procesador de NiFi espera encontrarla, activamos el grupo de procesadores para que realice las transformaciones tal y como las hemos dispuesto previamente:



Vemos como tarda un rato en proceder a la ingesta de datos, y cómo las tareas van atravesando los diferentes procesos, así como las propiedades del FlowFile que genera el procesador de transformación de formato que van apareciendo en los logs de NiFi:



```

a6-b96d-03d1c4425dca,ctatm=StandardContentctatm [resourcectatm=Stan
inner=default, section=1], offset=126, length=137],offset=0,name=000
-----
FlowFile Properties
Key: 'entryDate'
    Value: 'Fri May 30 14:40:34 CEST 2025'
Key: 'lineageStartDate'
    Value: 'Fri May 30 14:40:34 CEST 2025'
Key: 'fileSize'
    Value: '137'
FlowFile Attribute Map Content
Key: 'filename'
    Value: '000000_0'
Key: 'hadoop.file.url'
    Value: 'hdfs://localhost:9000/data/aggregations/000000_0'
Key: 'hdfs.group'
    Value: 'supergroup'
Key: 'hdfs.lastModified'
    Value: '1748608788824'
Key: 'hdfs.length'
    Value: '541'
Key: 'hdfs.owner'
    Value: 'bigdata'
Key: 'hdfs.permissions'
    Value: 'rw-r--r--'
Key: 'hdfs.replication'
    Value: '1'
Key: 'mime.type'
    Value: 'application/json'
Key: 'path'
    Value: '/data/aggregations'
Key: 'record.count'
    Value: '3'
Key: 'schema.name'
    Value: 'hive_schema'
Key: 'uuid'

```

Aquí vemos las propiedades del FlowFile generado donde se ve la ruta de entrada, el archivo parquet que procesa, el esquema que infiere de los datos y que emplea para escribir el Json de salida.

```

bigdata@Big-Data:~/confluent-7.5.1$ bin/kafka-console-consumer --bootstrap-server localhost:9092 --topic aggregations
[{"total_destinations":1,"country":"Indonesia"}, {"total_destinations":10,"country":"Kenya"}, {"total_destinations":23,"country":"Mexico"}]

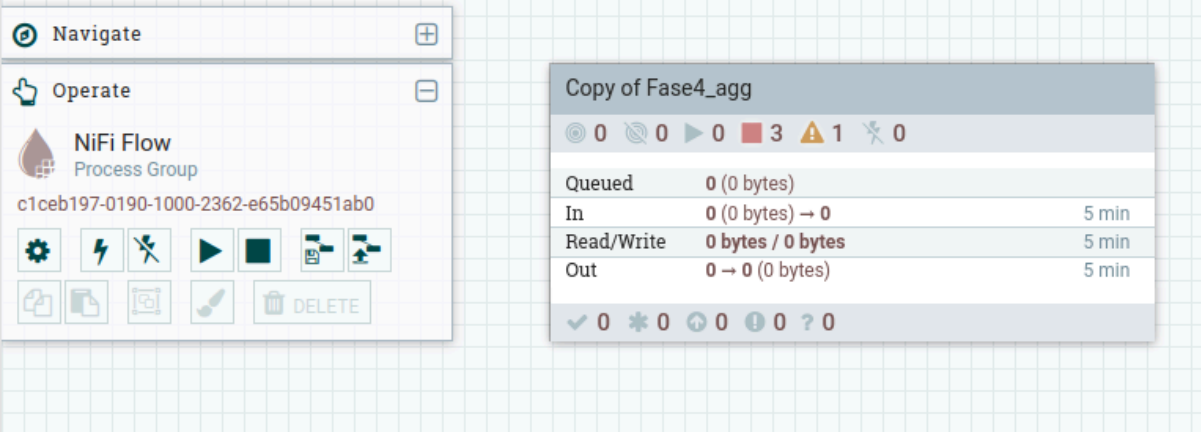
```

Aquí vemos como los datos que recibe Kafka constan de un único array de salida con los Jsons de cada una de las líneas que contenía la tabla 'aggregations'. Este sería el fin de la Fase 4 del proyecto, en los siguientes pasos vamos a realizar cambios en el grupo de procesos de NiFi para que la información se suministre línea a línea de la tabla, permitiéndonos además tener un consumidor escuchando de forma continua los datos recibidos a Kafka.

Actualización del flujo de datos de NiFi

Para comenzar la fase 5 debemos actualizar el grupo de procesadores de la fase previa, de modo que el array de archivos Jsons generador al leer los datos de la tabla almacenada en HDFS se suministren a Kafka uno a uno.

Para comenzar, en lugar de alterar el grupo de procesadores de la Fase 4, vamos a hacer una copia para mayor seguridad:

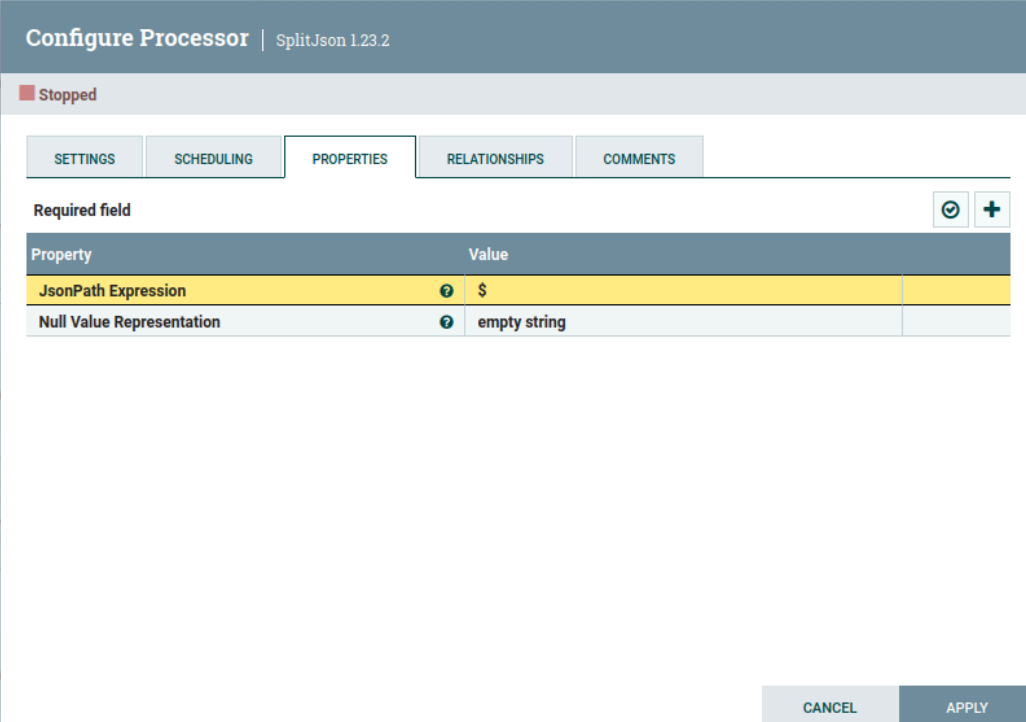


The screenshot shows the NiFi interface. On the left, a sidebar titled 'NiFi Flow' displays a 'Process Group' with ID 'c1ceb197-0190-1000-2362-e65b09451ab0'. It includes icons for settings, lightning bolt, scissors, play, stop, and other actions, along with a 'DELETE' button. On the right, a panel titled 'Copy of Fase4_agg' shows the status of the processor group. It includes a top bar with icons and counts: 0 (eye), 0 (stop), 0 (play), 3 (red square), 1 (yellow triangle), and 0 (scissors). Below this, a table displays the following data:

Queued	0 (0 bytes)	
In	0 (0 bytes) → 0	5 min
Read/Write	0 bytes / 0 bytes	5 min
Out	0 → 0 (0 bytes)	5 min

At the bottom of the panel, there is a row of icons and counts: a checkmark (0), a star (0), a play button (0), a red circle (0), a yellow circle (0), and a question mark (0).

Vamos a agregar entre la salida del procesador **FetchParquet** y **PublishKafka_2.6** un procesador nuevo que separe la lista que devuelve el primer procesador en archivos Json individuales y se los suministre uno a uno al publicador de Kafka. Este procesador se llama SplitJson, y lo implementamos con la configuración que se ve en la siguiente imagen:



The screenshot shows the 'Configure Processor' dialog for the 'SplitJson 1.23.2' processor. The dialog is titled 'Configure Processor | SplitJson 1.23.2' and has a 'Stopped' status indicator. It features five tabs: 'SETTINGS', 'SCHEDULING', 'PROPERTIES', 'RELATIONSHIPS', and 'COMMENTS'. The 'PROPERTIES' tab is currently selected. Below the tabs, there is a 'Required field' section with a table of properties. The table has two columns: 'Property' and 'Value'. The properties are as follows:

Property	Value
JsonPath Expression	\$
Null Value Representation	empty string

At the bottom right of the dialog, there are 'CANCEL' and 'APPLY' buttons.

The screenshot displays the Apache NiFi console interface. On the left, a sidebar contains a 'Navigate' button and an 'Operate' section with various tool icons. The main area shows a data flow diagram on a grid background. The flow begins with a 'ListHDFS' connector (red icon) which connects to a 'FetchParquet' connector (red icon with a warning triangle). From 'FetchParquet', the flow goes to a 'SplitJson' connector (red icon). The 'SplitJson' connector has two outgoing flows: one to a 'LogAttribute' connector (red icon) and another to a 'PublishKafka_2.6' connector (red icon). Both 'LogAttribute' and 'PublishKafka_2.6' have outgoing flows that converge back into the 'LogAttribute' connector. Each connector box displays its name, version (1.23.2), organization (org.apache.nifi), and performance metrics (In, Read/Write, Out, Tasks/Time). Intermediate flow labels like 'Name success' and 'Name failure, retry, success' are also visible. The top status bar shows the process group 'Fase5_agg' and its ID '01971000-8e87-1107-88b1-ea0df8af0025'.

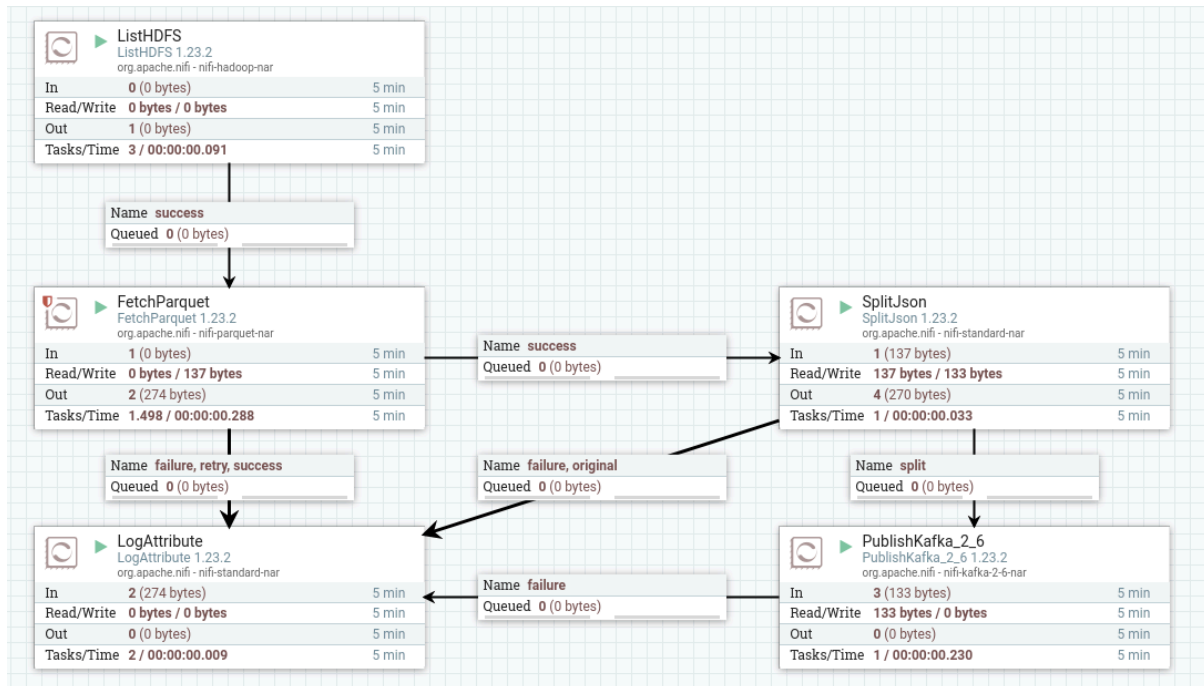
Para esta fase, el arranque es idéntico al de la fase 4, salvo por el hecho de que emplearemos un servidor de KSQL en lugar de arrancar Kafka. Para ello, en la ventana de confluen vamos a lanzar el comando 'confluent local services ksql-server start':

```
bigdata@Big-Data:~/confluent-7.5.1$ confluent local services ksql-server start
The local commands are intended for a single-node development environment only,
rrent/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.011580
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
ksqlDB Server is [UP]
bigdata@Big-Data:~/confluent-7.5.1$
```

A continuación entramos al servicio de KSQL, donde vamos a leer los datos procedentes de NiFi. Para ello, simplemente necesitamos el comando 'bin/ksql'.

Con esto ya dispuesto, vamos a iniciar el grupo de procesadores que hemos actualizado para esta fase:



En esta ocasión la secuencia de transformaciones requiere más tiempo que en la fase anterior, pero comprobando en la terminal de KSQL podemos ver cómo los datos solicitados van apareciendo, esta vez de forma independiente.

```

ksql> SELECT * FROM aggregations WHERE total_destinations >10 EMIT CHANGES;
+-----+-----+
|TOTAL_DESTINATIONS|COUNTRY|
+-----+-----+
|23                |Mexico |
+-----+-----+

```

Aparece únicamente Mexico, con un total de 23 aeropuertos a más de 5000 pies de altura que sirven de destino a rutas inactivas. Efectivamente de los resultados mostrados por la tabla final de la fase anterior éste es el único con un recuento total mayor a 10.

Para comprobar que el servidor de KSQL permanece a la escucha de los datos del topic de Kafka, vamos a añadir datos nuevos a la tabla. Para ello empleamos el siguiente comando:

```

0: jdbc:hive2://> INSERT INTO aggregations VALUES(41,"Germany");

```

Y unos segundos después vemos cómo los resultados en KSQL se actualizan:

```

ksql> SELECT * FROM aggregations WHERE total_destinations >10 EMIT CHANGES;
+-----+-----+
|TOTAL_DESTINATIONS|COUNTRY|
+-----+-----+
|23                |Mexico |
|41                |Germany|
+-----+-----+
Press CTRL-C to interrupt

```

Cierre de las aplicaciones ejecutadas

Para finalizar, vamos a cerrar todas las aplicaciones y herramientas desplegadas en este proyecto, tal como hicimos al finalizar los sprints previos.

Como se ve en la imagen previa, podemos terminar la query de KSQL pulsando 'Ctrl+C', y a continuación cerrar la herramienta introduciendo el comando 'exit'

```
ksql> exit
Exiting ksqlDB.
bigdata@Big-Data:~/confluent-7.5.1$
```

Y cerramos los servicios de confluent con el comando 'confluent local services stop', válido tanto si hemos arrancado Kafka únicamente como si hemos lanzado el KSQL-server.

```
bigdata@Big-Data:~/confluent-7.5.1$ confluent local services stop
The local commands are intended for a single-node development env
rrent/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.011580
Control Center is [DOWN]
Stopping ksqlDB Server
ksqlDB Server is [DOWN]
Connect is [DOWN]
Kafka REST is [DOWN]
Stopping Schema Registry
Schema Registry is [DOWN]
Stopping Kafka
Kafka is [DOWN]
Stopping ZooKeeper
ZooKeeper is [DOWN]
bigdata@Big-Data:~/confluent-7.5.1$
```

Por otra parte, paramos los grupos de procesos activos de NiFi desde la interfaz gráfica y desactivamos la herramienta empleando el comando 'bin/nifi.sh stop'.

```
bigdata@Big-Data:~/nifi-1.23.2$ bin/nifi.sh stop

Java home: /usr/lib/jvm/java-8-openjdk-amd64
NiFi home: /home/bigdata/nifi-1.23.2

Bootstrap Config File: /home/bigdata/nifi-1.23.2/conf/bootstrap.conf

2025-06-02 14:51:37,538 INFO [main] org.apache.nifi.bootstrap.Command
d Apache NiFi has accepted the Shutdown Command and is shutting down
now
2025-06-02 14:51:37,550 INFO [main] org.apache.nifi.bootstrap.Command
d NiFi PID [28476] shutdown in progress...
2025-06-02 14:51:39,559 INFO [main] org.apache.nifi.bootstrap.Command
d NiFi PID [28476] shutdown in progress...
2025-06-02 14:51:41,581 INFO [main] org.apache.nifi.bootstrap.Command
d NiFi PID [28476] shutdown completed

bigdata@Big-Data:~/nifi-1.23.2$
```


Por último, desactivamos el sistema de ficheros distribuido con el comando 'sbin/stop-dfs.sh'

```
bigdata@Big-Data:~/hadoop-3.3.6$ sbin/stop-dfs.sh
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [Big-Data]
bigdata@Big-Data:~/hadoop-3.3.6$
```

Y con esto concluimos todos los pasos necesarios para completar este proyecto, incluyendo tanto la fase obligatoria como la fase opcional.

Conclusiones

En este trabajo hemos visto el proceso completo para realizar agregaciones complejas cruzando datos de diferentes tablas a partir de ficheros en un sistema distribuido de archivos. Además, hemos desarrollado sendos flujos de datos para procesar dicha información y suministrarla a un consumidor de Kafka, generando por tanto un flujo ETL completo.

Además, en la fase opcional hemos mejorado la arquitectura previa para crear un proceso de lectura en streaming de datos, permitiendo un procesamiento posterior mediante KSQL, en una query que recibía datos desde un consumidor de kafka en tiempo real. En este caso estaríamos hablando de un flujo de tipo ETLT, dado que realizábamos una nueva query sobre los datos que recibía Kafka.

Este es sin duda un proceso de trabajo muy relevando en el desarrollo de sistemas de datos distribuidos, como pueda ser en la creación de Data Warehouse o Data Lakes que puedan ser empleados en grupos de trabajo deslocalizados o para el uso de aplicaciones que requieran de gestión de datos siguiendo principios de Big Data.